Konzeptdokument: Lyx Aerospace Extension (v2)

Autor: Manus AI **Datum:** 03. April 2026

1. Einleitung

Dieses Dokument beschreibt ein erweitertes Konzept für die “Lyx Aerospace Extension”, eine Spezifikation, die darauf abzielt, die Entwicklung von Software für kritische Raumfahrtanwendungen zu optimieren. Der Fokus liegt auf der Gewährleistung von **Deterministik**, **Hardware-Integration**, **formaler Sicherheit**, **robuster Telemetrie** und **Laufzeit-Integrität**. Das vorliegende Konzept integriert die ursprünglichen Anforderungen mit zusätzlichen Vorschlägen, die auf Best Practices und Industriestandards wie DO-178C, MISRA C und ECSS basieren, um ein umfassendes und zertifizierbares System zu ermöglichen.

2. Grundlagen der Lyx Aerospace Extension

Die Lyx Aerospace Extension konzentriert sich auf fünf Kernbereiche, um die Zuverlässigkeit und Sicherheit von Raumfahrtsoftware zu gewährleisten:

2.1. Flightoperations (Deterministik & Echtzeit)

Das Hauptziel in diesem Bereich ist die **absolute Vorhersagbarkeit der Programmausführung**, um strikte Zeitvorgaben (Hard Real-Time) zu erfüllen. Dies wird durch folgende Features erreicht:

- **WCET (Worst-Case Execution Time) Kontrolle:** Jede `while`-Schleife muss ein statisches Limit besitzen (`bounded_while`), um die maximale Ausführungszeit zu begrenzen. Die EBNF-Regel hierfür ist `WhileStmt := "while" "(" Expr ")" "limit" "(" IntLiteral ")" Block`.

- **Statischer Stack-Verbrauch:** Rekursion ist in `@flight_crit` Sektionen verboten. Dies verhindert Stack-Overflows in Umgebungen, in denen Debugging schwierig oder unmöglich ist, wie im tiefen Weltraum.
- **Echtzeit-Scheduling:** Native Prioritäts-Attribute für Funktionen sollen Priority Inversion verhindern und ein zuverlässiges Echtzeitverhalten sicherstellen.

2.2. Construction (Hardware-Integration & Integrität)

Schnittstellen zur physischen Welt müssen präzise und robust gegenüber extremen Umweltfaktoren wie Strahlung sein. Hierfür sind folgende Features vorgesehen:

- **Bit-Level Memory Mapping:** Ermöglicht die explizite Bit-Positionierung in Datenstrukturen. Ein Beispiel ist `struct Thruster @packed { power: u12 at(0); is_active: bool at(12); }`, um Bits 0-11 für `power` und Bit 12 für `is_active` zu definieren.
- **Triple Modular Redundancy (TMR):** Das Attribut `@redundant` weist den Compiler an, drei Kopien einer Variable im RAM zu erzeugen. Ein Majority-Vote-Algorithmus korrigiert beim Lesen Bit-Flips, die durch kosmische Strahlung verursacht werden können.

2.3. Mission-Handling (Formale Sicherheit)

Logische Fehlentscheidungen sollen durch die mathematische Beweisbarkeit des Codes vermieden werden. Dies wird durch folgende Mechanismen unterstützt:

- **Design by Contract:** Funktionen können mit `pre`- und `post`-Conditions versehen werden, um Vor- und Nachbedingungen der Ausführung zu definieren. Beispiel: `fn adjust_orbit(delta_v: f64) pre(delta_v > 0.1 && fuel > 10.0) post(orbit_stable == true)`.
- **Ranged Types (Einheiten-Sicherheit):** Typen können mit Wertebereichsbeschränkungen definiert werden, z.B. `type Identifier is BaseType range (Min .. Max)`. Dies verhindert physikalisch unmögliche Werte wie negative Drücke oder Kurse über 360°.

2.4. Telemetrie (Datenübertragung & Diagnose)

Die sichere Kommunikation zwischen Raumfahrzeug und Bodenstation unter Berücksichtigung unterschiedlicher Architekturen ist entscheidend:

- **Endianness-Management:** `@big_endian` / `@little_endian` Annotationen für Strukturen ermöglichen die automatische Konvertierung beim Senden von Daten zwischen Systemen mit unterschiedlicher Byte-Reihenfolge (z.B. PowerPC-Bordcomputer und x86-Bodenstationen).
- **Zero-Copy Networking:** `flat struct` Strukturen ohne Pointer können direkt bitweise in den Funk-Buffer geschoben werden, was die Effizienz der Datenübertragung erhöht.
- **Lock-free Buffering:** Ein nativer `RingBuffer<T>`-Typ erlaubt das Schreiben von Sensordaten in Interrupt-Handlern, ohne den Hauptprozess durch Locks zu blockieren.

2.5. Integritäts-Management-System (Nested Redundancy)

Um den laufenden Programmcode in Lyx v0.6.0 vor Umwelteinflüssen (z.B. Single Event Upsets durch Strahlung) zu schützen, wird ein dreistufiges Integritäts-Management-System eingeführt, das auf Definition, Überwachung und Wiederherstellung basiert.

2.5.1. Syntax-Erweiterung: `@integrity` Blöcke

Ein neues Pragma ermöglicht den Schutz ganzer Code-Bereiche (Module oder Funktionen).

```
IntegrityAttr  := "@integrity" "(" "mode" ":" IntegrityMode "," "interval" ":"
IntLiteral ")" ;
IntegrityMode  := "software_lockstep" | "scrubbed" | "hardware_ecc" ;
```

Beispiel:

```
@integrity(mode: scrubbed, interval: 100) // Prüft alle 100ms die Code-Integrität
unit navigation.core;

@critical_state
var thrust_vector: int64;

pub fn calculate_burn() {
    // Dieser Code wird im RAM durch Hashes geschützt
}
```

2.5.2. Das Metadaten-Layout (`.meta_safe` Section)

Der Compiler generiert für jedes geschützte Modul eine isolierte Sektion im ELF-Layout. Um einen "Single Point of Failure" zu vermeiden, wird das TMR-Hash-Prinzip angewendet.

- **Code-Range:** Start- und Endadresse des zu schützenden `.text`-Segments.
- **Triple-Hash-Store:** Drei Kopien des SHA-256 oder CRC32-Hashes, getrennt durch 4KB Padding, um physische Nachbarschaftseffekte (wie Rowhammer oder lokale Teilcheneinschläge) zu minimieren.
- **Recovery-Pointer:** Adresse des "Golden Image" im schreibgeschützten Flash/ROM.

2.5.3. Die Runtime-Überwachung (Der Scrubber)

Ein in die ELF-Binary eingebetteter Low-Level-Background-Task (Scrubber) überwacht die Integrität kontinuierlich.

1. **Hash-Voting:** Die Runtime liest die drei Hashes aus `.meta_safe`.
2. **Konsens:** Wenn `H1 == H2` oder `H1 == H3`, wird `H1` als Referenz genommen. Weicht ein Hash ab, wird er sofort im RAM durch die korrekten Kopien repariert (Self-Healing).
3. **Code-Validierung:** Die Runtime berechnet den aktuellen Hash des laufenden Codes im RAM und vergleicht ihn mit dem konsolidierten Referenz-Hash.
4. **Action:** Bei einer Diskrepanz wird die `panic`-Kaskade ausgelöst und das System kann aus dem "Golden Image" wiederhergestellt werden.

2.5.4. Speicher-Mapping & Physische Trennung

Der Backend-Emitter (v0.1.7) stellt sicher, dass redundante Kopien physisch getrennt gespeichert werden, sofern das Target-System dies unterstützt.

Komponente	Strategie	Umsetzung in Lyx
Befehlssatz	`@integrity(mode: software_lockstep)`	Der Compiler dupliziert Instruktionen und fügt `cmp`-Befehle zwischen logischen Blöcken ein [1].
Stack	Shadow Stack	Ein zweiter Stack-Pointer (z.B. `r15` statt `rsp`) führt eine Kopie der Rücksprungadressen, um Control-Flow-Manipulationen und Bit-Flips zu erkennen [2].
Heap	Parity-Alloc	`new` allokiert bei kritischen Klassen automatisch N Kopien mit Interleaving-Abstand.

2.5.5. Implementierungs-Priorität

1. **Phase 1:** Implementierung der `.meta_safe` Sektion im Compiler-Backend.
2. **Phase 2:** Integration der `VerifyIntegrity()` Builtin-Funktion (manuell oder timer-basiert aufrufbar).
3. **Phase 3:** Automatisches “Scrubbing” durch einen Hintergrund-Thread (z.B. via pthreads oder native Syscalls) [3].

3. Erweiterte EBNF-Vorschläge (Zusammenfassung)

Die vorgeschlagenen Erweiterungen der Lyx Aerospace Extension umfassen folgende EBNF-Regeln:

```
 (* Erweiterte Attribute *)
Annotation    := "@flight_crit" | "@redundant" | "@packed" | "@no_heap"
               | "@big_endian" | "@little_endian" | IntegrityAttr

(* Integritäts-Management *)
IntegrityAttr := "@integrity" "(" "mode" ":" IntegrityMode "," "interval" ":"
IntLiteral ")"
IntegrityMode := "software_lockstep" | "scrubbed" | "hardware_ecc"

(* Erweiterte Typen *)
RangedType    := PrimitiveType "range" "(" ConstExpr ".." ConstExpr ")"
FieldDecl     := Identifier ":" Type [ "at" "(" BitOffset ")" ] ";"

(* Verträge für Funktionen *)
Contract      := { "pre" "(" Expr ")" | "post" "(" Expr ")" }
FuncDecl      := [Annotation] "fn" Identifier "(" Params ")" [":" Type]
                 [Contract] Block

(* Bounded Loops *)
WhileStmt     := "while" "(" Expr ")" "limit" "(" IntLiteral ")" Block
```

4. Die “Goldene Regel” für Lyx Aerospace

Zur Erleichterung der Zertifizierung sollte Lyx einen “Safe-Dialekt” anbieten, der folgende Einschränkungen durchsetzt:

- **Verbot dynamischer Allokation (`new`)** nach der Initialisierungsphase.
- **Ersetzung von Pointer-Arithmetik** durch sichere References.
- **Komplettes Unterbinden impliziter Typkonvertierungen (Casting)**, um Genauigkeitsverluste (z.B. `Float -> Int`) explizit sichtbar zu machen.

5. Identifizierte Lücken und Ergänzungsvorschläge

Basierend auf der Analyse der ursprünglichen Spezifikation und der Recherche zu Industriestandards wurden folgende Bereiche als potenzielle Lücken identifiziert und mit Ergänzungsvorschlägen versehen:

5.1. Fehlerbehandlung (Error Handling)

Ein robustes System für die Raumfahrt muss in der Lage sein, Fehler sicher zu erkennen und darauf zu reagieren. Die aktuelle Spezifikation enthält keine expliziten Mechanismen hierfür.

- **Vorschlag:** Einführung von `panic` **Strategien** für `@flight_crit` Sektionen, die beispielsweise einen automatischen Reboot in einen Safe-Mode auslösen können. Dies ist entscheidend für die **Fehlertoleranz** in kritischen Systemen [4]. Darüber hinaus sollte ein **definierter Fehlerzustandsautomat** implementiert werden, der das System bei nicht behebbaren Fehlern in einen sicheren Zustand überführt.

5.2. Speichermanagement (Memory Management)

Das Verbot von Heap-Allokation (`@no_heap`) ist ein guter Schritt, aber die Verwaltung statischer Ressourcen muss klar definiert sein.

- **Vorschlag:** Implementierung von **statischen Speicherpools** oder **"Region-based Memory Management"**. Dies ermöglicht eine effiziente und vorhersagbare Speicherverwaltung ohne die Risiken dynamischer Allokation. Solche Ansätze sind in sicherheitskritischen Systemen weit verbreitet, um die **Speicherintegrität** zu gewährleisten [5].

5.3. Concurrency & Synchronisation

Obwohl Lock-free Buffering für Telemetrie vorgesehen ist, fehlen umfassende Konzepte für die Verwaltung von Nebenläufigkeit, Task-Prioritäten und Interrupt-Latenzen.

- **Vorschlag:** Einführung eines `@interrupt_handler` **Attributs** zur Kennzeichnung von Interrupt-Service-Routinen und Mechanismen zur **Prioritäts-Vererbung** oder **Prioritäts-Ceiling-Protokollen**, um Deadlocks und Priority Inversion in Multitasking-Umgebungen zu verhindern. Dies ist ein Kernaspekt von **Echtzeitbetriebssystemen (RTOS)**, die in der Raumfahrt eingesetzt werden [6].

5.4. Zertifizierungs-Unterstützung (Compliance)

Die Zertifizierung von Raumfahrtsoftware nach Standards wie DO-178C und ECSS erfordert eine lückenlose Rückverfolgbarkeit (Traceability) von Anforderungen bis zum Code.

- **Vorschlag:** Implementierung eines `@requirement("ID")` **Attributs**, das Code-Abschnitte direkt mit spezifischen Anforderungen verknüpft. Dies erleichtert die **Traceability Matrix** und die Verifikation gemäß **DO-178C** [7] und **ECSS-E-ST-40C** [8]. Zusätzlich sollten Metriken zur Code-Qualität und Testabdeckung, wie sie von **MISRA C** empfohlen werden, in den Entwicklungsprozess integriert werden, um die Einhaltung von **ECSS-Q-ST-80C** zu unterstützen [9].

5.5. I/O & Peripherie

Die Spezifikation adressiert die Hardware-Integration auf Bit-Ebene, aber standardisierte Schnittstellen für gängige Raumfahrt-Busse fehlen.

- **Vorschlag:** Definition von **standardisierten Treiber-Interfaces** für gängige Raumfahrt-Kommunikationsprotokolle wie **MIL-STD-1553** und **SpaceWire**. Dies fördert die Wiederverwendbarkeit und Interoperabilität von Hardware-Komponenten und Treibern [10].

5.6. Testing & Simulation

Für die Verifikation von Raumfahrtsoftware sind umfassende Test- und Simulationsmöglichkeiten unerlässlich.

- **Vorschlag:** Unterstützung für **Hardware-in-the-Loop (HIL)** und **Software-in-the-Loop (SIL)** Simulationen. Dies kann durch die Bereitstellung von `mock` **oder** `simulate` **Blöcken** für Hardware-Strukturen im Sprachdesign erreicht werden, um das Verhalten von Sensoren und Aktuatoren zu emulieren und den Code unter realistischen Bedingungen zu testen [11].

5.7. Floating Point Determinismus

Die Verwendung von Gleitkommazahlen kann auf verschiedenen Architekturen zu geringfügig unterschiedlichen Ergebnissen führen, was in sicherheitskritischen Anwendungen inakzeptabel ist.

- **Vorschlag:** Einführung eines **strikten Floating-Point-Modells** für `@flight_crit` Sektionen, das eine **garantierte Reproduzierbarkeit** von Gleitkommaoperationen über verschiedene Plattformen hinweg sicherstellt. Dies kann durch die Einhaltung spezifischer IEEE 754-Profile oder durch die Verwendung von Festkomma-Arithmetik in kritischen Bereichen erreicht werden [12].

5.8. Sicherheitsaspekte (MISRA C)

Die "Goldene Regel" adressiert bereits einige Sicherheitsaspekte. Eine explizite Integration von MISRA C-ähnlichen Regeln in den Sprachstandard würde die Code-Qualität und Sicherheit weiter erhöhen.

- **Vorschlag:** Integration von **MISRA C-Richtlinien** direkt in den Compiler oder als Teil des Sprachstandards, insbesondere in Bezug auf die Vermeidung von undefiniertem Verhalten, die Nutzung von Zeigern und die Typkonvertierung. Dies würde die Entwicklung von robustem und sicherem Code weiter fördern und die Einhaltung von Industriestandards erleichtern [13].

6. Fazit und Nächste Schritte

Die "Lyx Aerospace Extension" bietet mit der Integration des Integritäts-Management-Systems eine "festungsgleiche" Grundlage für die Entwicklung hochzuverlässiger Software in der Raumfahrt. Der minimale Overhead für das Dreifach-Hashing der Metadaten steht in einem exzellenten Verhältnis zum massiven Sicherheitsgewinn im Deep-Space-Einsatz. Durch die weitere Integration der vorgeschlagenen Ergänzungen, insbesondere in den Bereichen Fehlerbehandlung, Speichermanagement, Concurrency, Zertifizierung, I/O, Testing und Floating Point Determinismus, kann das Konzept weiter gestärkt und an die strengen Anforderungen der Raumfahrtindustrie angepasst werden. Die Einhaltung etablierter Standards wie DO-178C, ECSS und MISRA C ist dabei von größter Bedeutung.

Nächste Schritte:

1. Detaillierte Ausarbeitung der vorgeschlagenen Features und deren Integration in die EBNF-Spezifikation.
2. Entwicklung eines Prototyps oder einer Referenzimplementierung zur Validierung der Konzepte (insbesondere `.meta_safe` und Scrubber).

3. Abstimmung mit relevanten Stakeholdern und Experten aus der Raumfahrtindustrie.

7. Referenzen

[1] Doran, H. D. “Dynamic Lockstep Processors for Applications with…” arXiv, 2021. https://arxiv.org/pdf/2107.08997 [2] Zou, C. “Buddy Stacks: Protecting Return Addresses with Efficient…” ACM, 2022. https://dl.acm.org/doi/full/10.1145/3494516 [3] NASA. “IV&V Lessons Learned From a Memory-Scrub Anomaly.” https://www.nasa.gov/wp-content/uploads/2016/10/ivv_mirfakhraie_ivv_lessons_learned_from_a_memory-scrub_anomaly_090114.pdf [4] Parasoft. “DO-178C Software Compliance for Aerospace & Defense.” https://www.parasoft.com/learning-center/do-178c/overview/ [5] LDRA. “DO-178C Certification - Your Complete Verification Journey.” https://ldra.com/do-178/ [6] Ansys. “An introduction to space software standards ECSS-E-ST-40 and …” https://innovationspace.ansys.com/knowledge/forums/topic/an-introduction-to-space-software-standards-ecss-e-st-40-and-ecss-q-st-80c/ [7] RTCA. “DO-178() Software Standards Documents & Training.” https://www.rtca.org/do-178/ [8] ECSS. “ECSS-E-ST-40C Rev.1 – Software (30 April 2025).” https://ecss.nl/standard/ecss-e-st-40c-rev-1-software-30-april-2025/ [9] ECSS. “ECSS-Q-ST-80C Rev.2 (30 April 2025).” https://ecss.nl/wp-content/uploads/2025/05/ECSS-Q-ST-80C-Rev.2(30April2025).pdf [10] Bugseng. “Aerospace.” https://www.bugseng.com/aerospace-overview/ [11] Adacore. “Formal methods for Aerospace Applications.” https://www.adacore.com/blog/formal-methods-practice-and-theory [12] Adesso. “Programming Languages in Safety-Critical Applications.” https://www.adesso.de/en/news/blog/programming-languages-in-safety-critical-applications.jsp [13] MISRA. “MISRA C.” https://www.misra.org.uk/misra-c/